

AXLERO INNOVATIVE SOLUTIONS

API SENTINEL

A Runtime BOLA and Shadow API Detection Engine

Domain: API Security and Zero Trust Architecture

Duration of Project: July to August

Abstract

- APIs now move most of the data between modern services, and this has created a security gap that traditional firewalls were never built to close.
- Web Application Firewalls check payloads for malformed content, but they cannot recognise a technically valid request that quietly accesses another user's data.
- API Sentinel is a runtime security engine that observes traffic at the Linux kernel level using eBPF and reconstructs every HTTP request outside the application itself.
- It detects Broken Object Level Authorization (BOLA) , Broken Function Level Authorization (BFLA) , enumeration, bot and rate abuse, and undocumented shadow endpoints.
- High risk flows can be blocked directly at the kernel level, and all findings are shown on a live dashboard.
- This report covers the problem being addressed, the architecture and modules built, the testing done across four weekly phases, and the results, including a measured overhead below the five millisecond target.

1. Introduction

- Applications that once ran as a single unit are now split into many independent services that communicate through APIs.
- Traffic between these services often carries more sensitive data than what a user ever sees directly.
- API first products have made this more significant, since the API is frequently the main interface offered to customers and partners.
- Security tooling has not kept pace with this shift. Most existing defences focus on recognising bad input rather than bad intent.
- A request with correct syntax and a valid token, asking for an object that does not belong to the requester, is usually treated as legitimate by conventional tools.
- API Sentinel addresses this blind spot using kernel level visibility, so detection does not depend on the application logging the right information.

2. Problem Statement

- Web Application Firewalls are effective against malformed payloads but cannot recognise business logic abuse such as BOLA or BFLA .
- Shadow and zombie APIs, meaning endpoints that exist in production but were never documented or have been forgotten, remain invisible to most monitoring tools.
- Detection that depends on application level logging or SDK integration requires code changes and is often skipped under delivery pressure.
- These gaps map directly onto several categories in the OWASP API Security Top 10, which was used as the guiding framework for this project.

3. Objectives

- Capture and reconstruct HTTP traffic at the kernel level without requiring any change to application code.
- Detect BOLA and BFLA in real time using behavioural baselines rather than fixed rules alone.
- Discover shadow APIs by comparing live observed traffic against the application's documented specification.
- Keep the additional latency introduced by the monitoring pipeline under five milliseconds per request.
- Present all findings through a live dashboard that reflects the detection engine's output directly.

4. System Architecture

- API Sentinel is organised as a five stage pipeline.
- A client request reaches the target FastAPI application, and the same request is independently intercepted at the kernel level using eBPF.
- The captured event is handed to a Rust userspace application, which forwards structured JSON to a Python detection engine.
- The detection engine evaluates each request for risk. The output feeds a discovery pipeline that maintains a live API inventory and a dashboard API that serves the React frontend.
- A single feedback path connects the detection engine back to the eBPF layer, allowing a flow to be blocked at the kernel level once its risk score crosses a defined threshold.

Stage	Component	Responsibility
1	Kernel Capture	eBPF program attached to kprobes intercepts request and response traffic without touching application code
2	Userspace Loader	Rust application reads the kernel ring buffer and converts raw events into structured JSON
3	Detection Engine	Python modules parse each request, resolve an identity, and run authorization and abuse checks
4	Discovery Pipeline	Builds a live endpoint inventory and compares it against the documented API specification
5	Dashboard	React frontend consuming the Dashboard API to display traffic, alerts, and API posture

5. Methodology and Key Modules

The implementation is organised into four modules, each corresponding to one stage of the pipeline described above. Every module was built and tested independently before being integrated with the others.

5.1 Kernel Level Sniffer

- Written in C for the eBPF program and Rust for the userspace loader.
- Two kernel functions are instrumented using (kprobes), (tcp_recvmmsg) for incoming requests and (tcp_sendmsg_locked) for outgoing responses.
- Captured events pass through a ring buffer to the Rust application, which converts them into structured JSON.
- The same module owns the enforcement path, a pinned BPF map that the kernel checks on every packet, allowing a flagged flow to be dropped on command from the detection engine.

5.2 Detection Engine

- Decodes each captured payload and reconstructs the HTTP request.
- Resolves a consistent identity for the requester from the Authorization header, a client identifier header, or the source IP address.
- Runs each request through checks for HTTP method validity, missing authorization, sensitive path access, object enumeration, and function level authorization.
- A separate BOLA engine baselines the first object an identity accesses and tracks distinct foreign objects touched afterward, catching patterns that do not follow a simple sequence.
- A rate limiter tracks request volume per identity and endpoint to flag bot and business flow abuse.
- Every check adds to a shared risk score, and crossing the enforcement threshold triggers a kernel level block.

5.3 Shadow API Discovery Engine

- Consumes the same validated request stream produced by the detection engine.
- Normalises paths into templates so that different object identifiers on the same route are recognised as one endpoint.
- Builds a live inventory from this traffic and converts it into a standard OpenAPI document.
- Compares the observed document against the application's actual documented specification.
- Surfaces shadow endpoints that receive traffic but were never documented, and documented endpoints that have never been used.
- Writes results atomically so the dashboard never reads a partially written report.

5.4 Threat Dashboard

- Built with React and Vite, reading exclusively from the Dashboard API with no hardcoded data.
- Presents six views: a real time overview, an analytics view of attack patterns, a discovery view of API posture, an alerts feed, a masked log view, and a read only settings view.

6. Implementation Timeline

Work followed the four week structure defined in the internship handbook for Project 1 of the Month 1 cycle. Each week corresponds to one implementation phase, with a Mid Review covering weeks one and two and a Final Review covering weeks three and four.

Week	Phase	Focus
Week 1	Implementation Phase 1	Kernel level traffic capture, eBPF and Rust pipeline, JSON event generation
Week 2	Implementation Phase 2	Request parsing, identity resolution, authorization and BOLA detection
Week 3	Implementation Phase 3	Rate abuse detection, kernel level enforcement, discovery pipeline
Week 4	Implementation Phase 4	Dashboard integration, PII masking, performance validation

Technology Stack

Layer	Technology
Kernel capture	eBPF using kprobes, a ring buffer, and a pinned BPF map
Userspace loader	Rust with libbpf-rs
Detection engine	Python
Backend API	FastAPI and Uvicorn
Dashboard	React, Vite, and Recharts

7. Results and Testing

Each weekly phase closed with its own validation pass, and the complete pipeline was load tested using ApacheBench at the mid project checkpoint with 1000 requests at a concurrency of 10. The comparison below shows the baseline FastAPI application against the same endpoints with the full capture and detection pipeline active.

Endpoint	Baseline	With API Sentinel	Overhead	Result
GET users by id	11.428 ms	15.576 ms	4.148 ms	Pass
GET orders by id	13.035 ms	15.779 ms	2.744 ms	Pass
POST login	13.498 ms	13.851 ms	0.353 ms	Pass
POST register	10.665 ms	15.474 ms	4.809 ms	Pass

All four endpoints stayed under the five millisecond overhead target set at the start of the project.

8. Challenges Faced

- Debugging at the kernel level required careful handling of kprobe entry and return pairs, since the response payload for (tcp_recvmmsg) is only available once the kernel completes the receive.
- Tuning the BOLA baseline threshold to avoid false positives took several iterations, since ordinary multi object access by a legitimate user needed to remain distinct from enumeration behaviour.
- Ensuring PII masking ran before logging, rather than after, required restructuring the order of operations in the detection pipeline.
- Coordinating four independent services, the target application, the Rust loader, the Python engine, and the React dashboard, for consistent local testing added setup overhead during early weeks.

9. Conclusion

API Sentinel shows that effective API security can be achieved without any changes to application code. Observing traffic directly at the kernel level was sufficient to detect Broken Object Level Authorization, enumeration, rate abuse, and shadow endpoints in real time. High risk flows were blocked automatically at the kernel level without manual intervention, and the system maintained latency under the five millisecond target throughout testing. The completed work meets the objectives defined for Project 1 under the Axler Solutions internship guidelines, with all four implementation phases built and tested successfully.

10. References

- OWASP API Security Top 10
- Linux Kernel BPF and eBPF Documentation
- libbpf-rs Documentation
- FastAPI Documentation
- Uvicorn Documentation
- React Documentation
- Recharts Documentation
- ApacheBench Documentation